

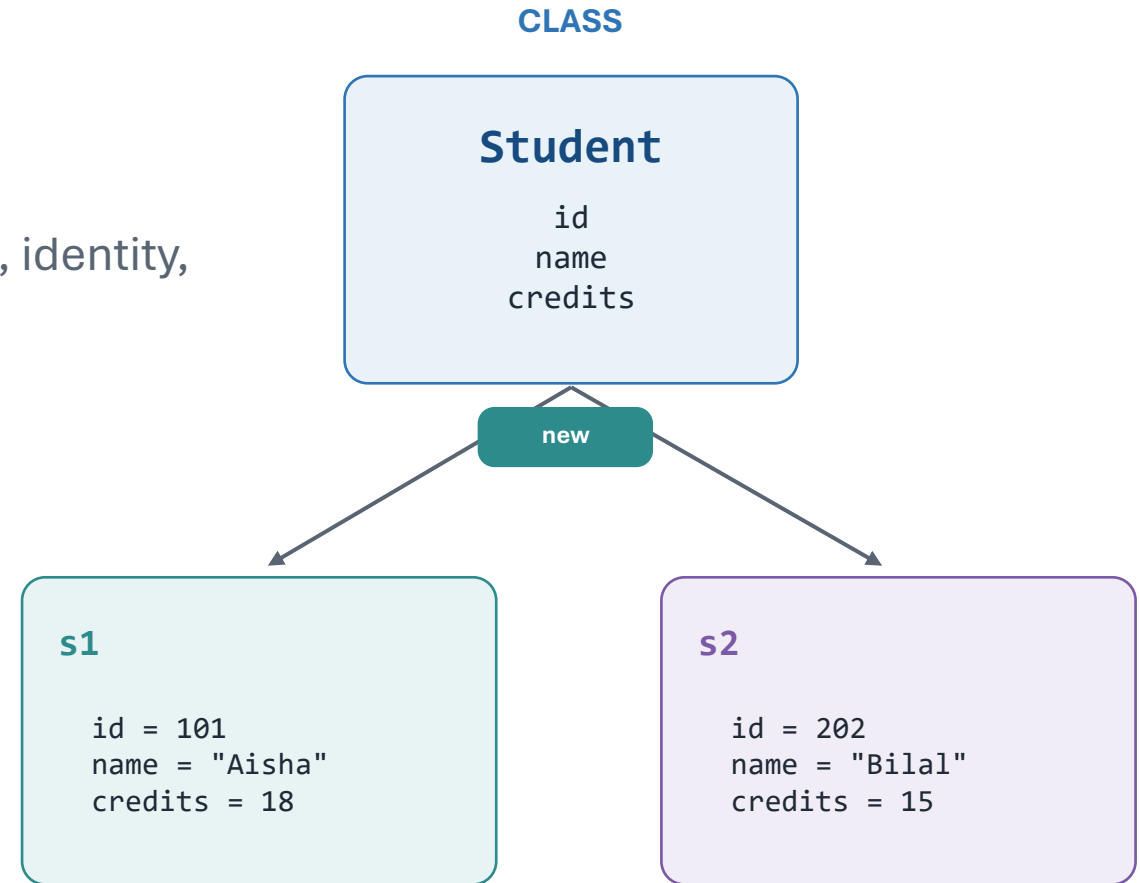
Classes & Objects

From Java source code to objects with state, behavior, identity, and references

CSC241 — Object-Oriented Programming

Fall 2026 • COMSATS University Islamabad

Instructor: Muhammad Shahid Bhatti

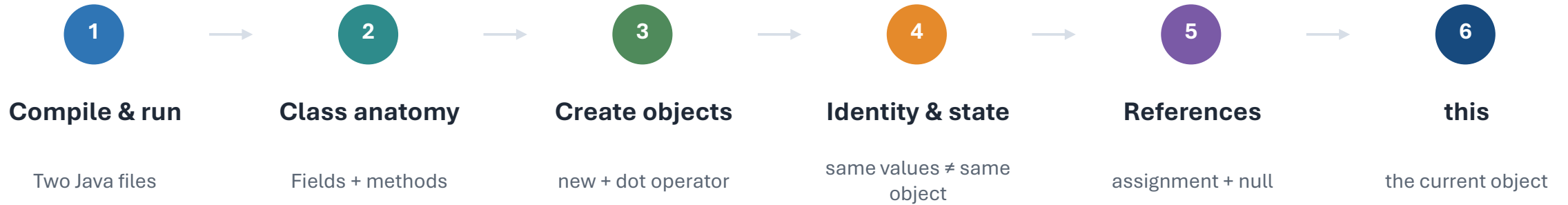


one blueprint → many independent objects

Lecture roadmap

ROADMAP

A live-coding lecture: we will repeatedly move between code, compiler feedback, and an object-level mental model.



Teaching rhythm

Explain → predict → live-code → compile → inspect output → modify → recompile. Students should see that OO concepts are not vocabulary; they are visible in executable code.

Learning outcomes

By the end of this lecture, students should be able to...

- Describe the anatomy of a simple Java class and distinguish fields from methods.
- Instantiate multiple objects from one class and access members using the dot operator.
- Explain class vs. instance, object state vs. object identity, and why equal-looking objects can still be distinct.
- Trace reference assignment, aliasing, reassignment, and null references.
- Use `this` to refer to the current object and resolve a field/parameter name collision.
- Compile and run a small multi-file Java program from the command line and interpret common compiler/runtime errors.

Bridge from Lecture 2

Last lecture ended with the Java source-to-execution pipeline. Today we zoom inside the program being compiled.



Lecture 2 question

How does Java execute my program?

Lecture 3 question

What are the objects that my program creates and manipulates?

Live coding setup

LIVE CODING

Start simple: one folder, a terminal, and a plain-text editor or IDE.

PROJECT FOLDER

```
oop-lecture03/  
  Student.java  
  Demo.java
```

TERMINAL

```
> javac Student.java Demo.java  
> java Demo
```

LIVE CODING

In class

Create the folder with students. Keep the terminal visible. Ask them what files they expect to appear after compilation before running javac.

Expected after compilation

Student.class

Demo.class

Source files are for humans + compiler. Class files contain JVM bytecode.

The compile-run cycle with multiple classes

A Java application can contain many classes. `javac` compiles source; `java` starts the class containing main.

Student.java

```
class Student {  
    int id;  
    String name;  
}
```

Demo.java

```
public class Demo {  
    public static void  
    main(String[] args) {  
  
        System.out.println("Lecture 3");  
    }  
}
```

Compile

```
javac Student.java Demo.java
```

Run the application entry point

```
java Demo
```

JVM loads the required .class files

Demo.class is started; Student.class is loaded when needed by the running program.

Compiler feedback is part of the learning loop

A compile error means the compiler cannot translate the source into valid bytecode. Read the first useful diagnostic.

INTENTIONALLY BROKEN

```
class Student {  
    int id  
    String name;  
}
```

javac output

```
Student.java:2: error: ';' expected  
    int id  
      ^
```

A productive debugging habit

- 1 Read the file + line number.
- 2 Read what the compiler expected.
- 3 Inspect that line and the line immediately before it.
- 4 Fix one issue, compile again.

Classroom move

Do not immediately fix the error. Ask students to interpret the diagnostic first.

Our first object-oriented program

The class describes a kind of object. The demo creates and uses one object.

Student.java

```
class Student {  
    int id;  
    String name;  
  
    void printSummary() {  
        System.out.println(id + " - " + name);  
    }  
}
```

Demo.java

```
public class Demo {  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        s1.id = 101;  
        s1.name = "Aisha";  
        s1.printSummary();  
    }  
}
```

Predict the output before running.

Anatomy of a class

A class groups the data that describes an object and the operations that object can perform.

```
class Student {                                // class declaration
    int id;                                    // field
    String name;                               // field

    void printSummary() {                      // method
        System.out.println(id + " - " + name);
    }
}
```

Class name **Student**

Names the new type.

Fields **id, name**

Hold object state.

Method **printSummary()**

Defines behavior.

Mental model

A class is a description/type. An object is a concrete runtime instance created from that description.

Class declaration: defining a new type

The class name becomes a type that can be used for reference variables.

```
class Student {  
    // members go here  
}
```

For today

We keep the declaration deliberately simple.
Access modifiers and encapsulation are handled
in a later lecture.

class **Student** { ... }

- **class** tells Java that we are defining a class.
- **Student** is the type name chosen by the programmer.
- The braces contain fields, methods, constructors, and other members.
- Convention: Java class names use UpperCamelCase.

Fields represent object state

State is the set of data values currently held by an object.

```
class Student {  
    int id;  
    String name;  
    int completedCredits;  
}
```

Ask students

Which of these fields are likely to change during a semester? Which should probably never change after creation?

Student object: s1

id	101
name	"Aisha"
completedCredits	18

State can change while the object remains the same object.

Methods represent object behavior

A method is an operation associated with the object. It can read or change object state.

```
class Student {  
    int completedCredits;  
  
    void addCredits(int amount) {  
        completedCredits = completedCredits +  
amount;  
    }  
  
    void printCredits() {  
        System.out.println(completedCredits);  
    }  
}
```

addCredits(3)



completedCredits

18 → 21

Behavior often exists to query or safely change state.

The OO shift: state and behavior belong together

Instead of passing scattered data to unrelated functions, the object becomes responsible for operations on its own state.

Procedure-centered view

studentId
studentName
studentCredits

addCredits(studentCredits, 3)
printStudent(studentId, studentName,
studentCredits)

Object-centered view

Student s1

s1.addCredits(3)
s1.printSummary()

The receiver object determines which state
the method operates on.

Live coding checkpoint 1 — build the Student class

[LIVE CODING](#)

Type it with the students rather than pasting it. Compile after each small change.

Student.java

```
class Student {  
    int id;  
    String name;  
    int completedCredits;  
  
    void printSummary() {  
        System.out.println(id + " | " + name  
            + " | credits=" + completedCredits);  
    }  
}
```

[LIVE CODING](#)

Suggested sequence

- 1 Create empty class.
- 2 Add one field → compile.
- 3 Add remaining fields → compile.
- 4 Add method → compile.
- 5 Introduce one error intentionally.
- 6 Fix and compile cleanly.

Why compile incrementally?

Students learn to associate a small source edit with its compiler consequence instead of debugging a large block all at once.

Instantiating an object

A class definition does not create Student objects by itself. new creates an instance at runtime.

```
Student s1 = new Student();
```

Student

reference type

s1

reference variable

=

new Student()

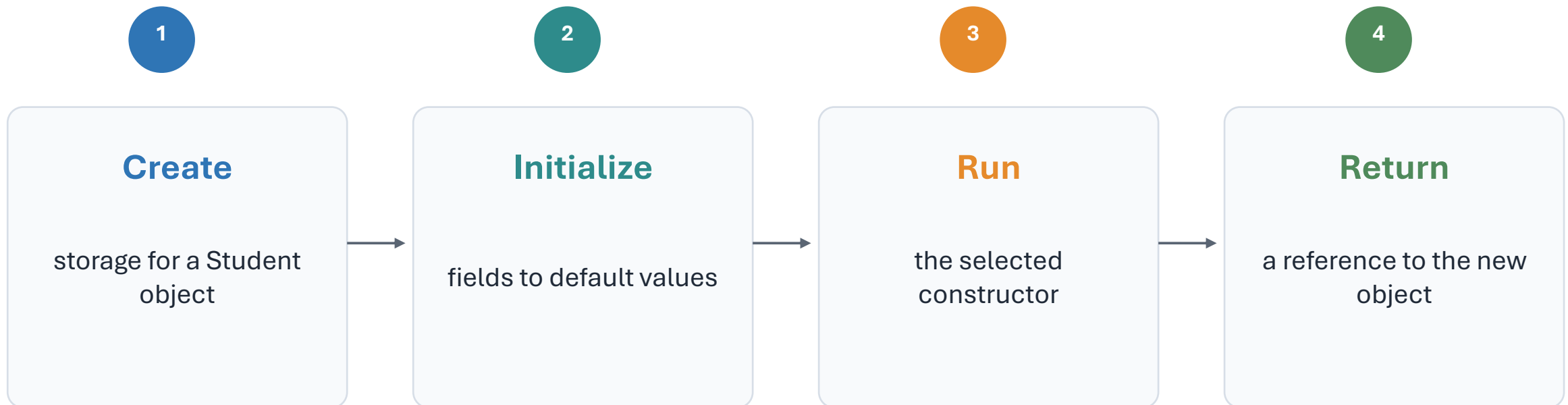
create a Student object

Small preview

Because Student declares no constructor yet, Java provides an implicit no-argument constructor. Constructors are covered in detail later.

What does new Student() mean conceptually?

We keep the memory details light today; the goal is the object-creation mental model.



Default field values

For a newly created object, numeric fields begin at 0, boolean at false, and reference fields at null—unless initialization code changes them.

Using the dot operator

The expression before the dot identifies the receiver object. The member after the dot identifies what we access or invoke.

```
s1.id = 101;  
s1.name = "Aisha";  
s1.completedCredits = 18;  
s1.printSummary();
```

receiver member

```
s1 . printSummary()
```

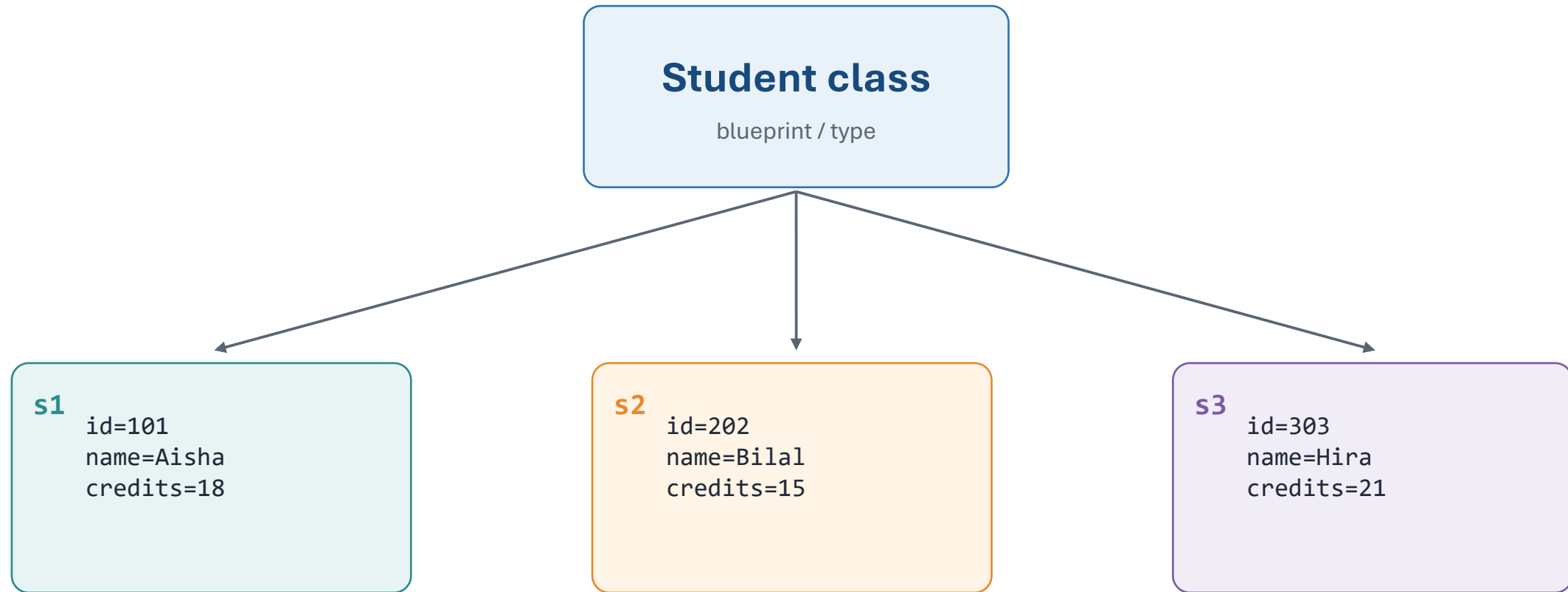
**“Ask object s1 to execute its
printSummary behavior.”**

Prediction question

What changes if we replace s1 with s2? The same method code executes, but it operates on a different receiver object and therefore different state.

Class vs. instance

One class can be used to create many objects. Each object has its own instance state.



The class definition is shared; the field values belong to individual objects.

Multiple objects from the same class

The same member names exist in every instance, but the values are independent.

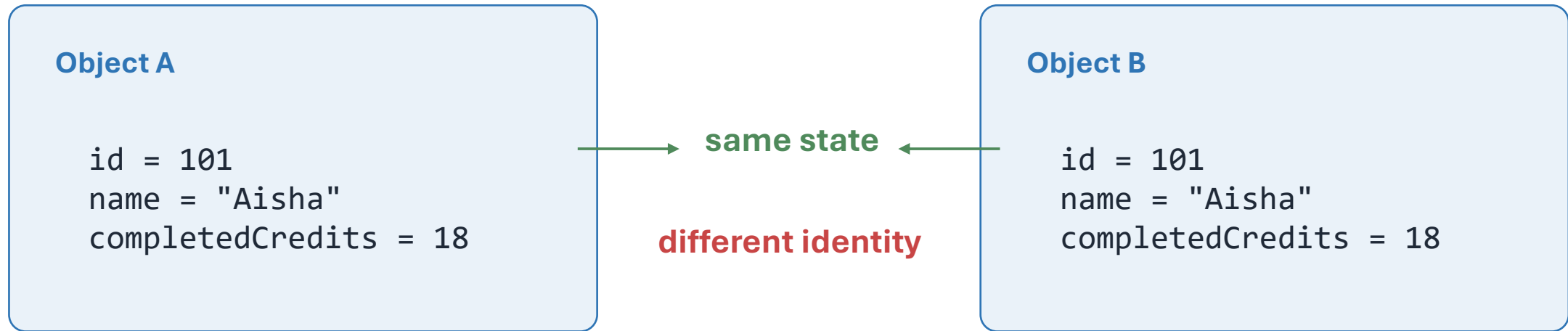
```
Student s1 = new Student();  
s1.id = 101;  
s1.name = "Aisha";  
  
Student s2 = new Student();  
s2.id = 202;  
s2.name = "Bilal";  
  
s1.printSummary();  
s2.printSummary();
```

s1**101 | Aisha****s2****202 | Bilal**

Change s1.name. Does s2.name change?

Object identity vs. object state

Two objects can contain exactly the same field values and still be two distinct objects.



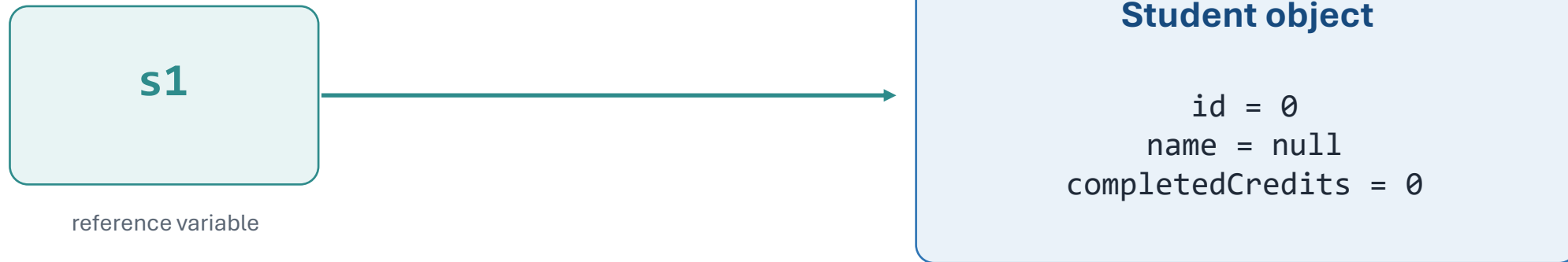
Important distinction

State answers “what values does the object hold now?” Identity answers “which specific object is this?”

Reference variables do not contain the object itself

A variable of class type stores a reference that can designate an object.

```
Student s1 = new  
Student();
```

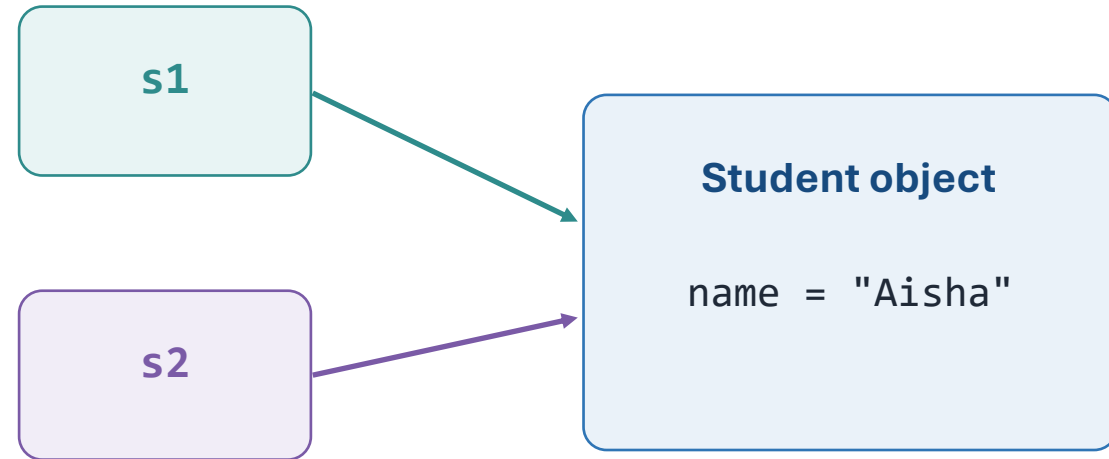


Think “s1 refers to a Student object,” not “s1 is the object.”

Reference assignment creates another reference to the same object

Assignment copies the reference value. It does not automatically clone the object.

```
Student s1 = new Student();  
s1.name = "Aisha";  
  
Student s2 = s1;
```



Alias

s1 and s2 are aliases: two reference variables that designate the same object.

Live coding checkpoint 2 — prove aliasing by mutation

LIVE CODING

Do not only explain the diagram. Make students predict the output, then run it.

```
Student s1 = new Student();  
s1.name = "Aisha";  
  
Student s2 = s1;  
s2.name = "Ayesha Khan";  
  
System.out.println(s1.name);  
System.out.println(s2.name);
```

LIVE CODING

Prediction

What does s1.name print?

Why?

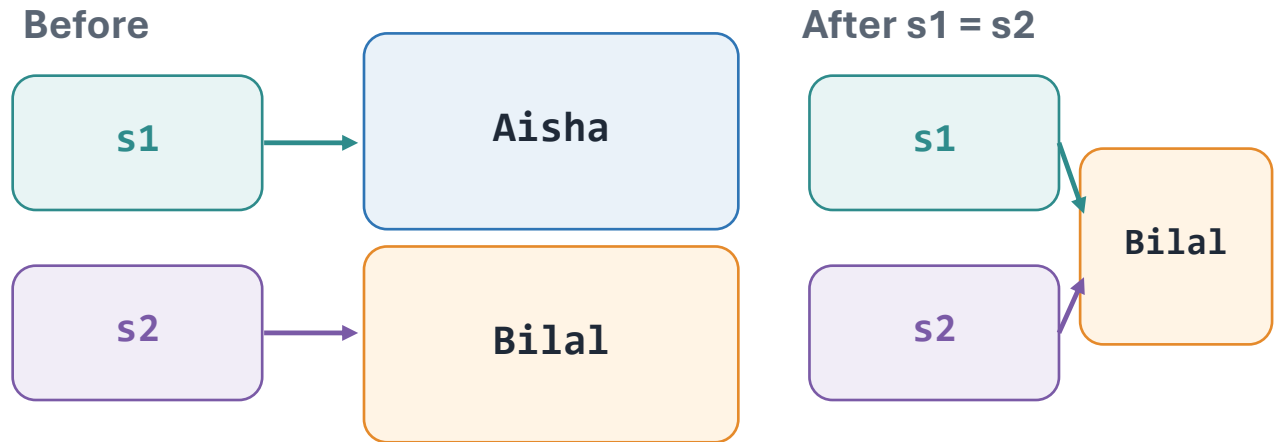
After running

Change s2 = new Student(); and rerun. The change creates a second object and breaks the aliasing relationship.

Reassigning a reference changes which object the variable designates

The old object is not modified merely because a reference variable is pointed elsewhere.

```
Student s1 = new Student();  
Student s2 = new Student();  
  
s1.name = "Aisha";  
s2.name = "Bilal";  
  
s1 = s2;
```



Preview

What happens to the `Aisha` object if no references point to it? We return to object lifetime and garbage collection later in the course.

null means “no object reference”

A reference variable can deliberately contain null to represent the absence of an object.

```
Student currentStudent =  
null;
```

currentStudent ----- no object

null is a valid value for a reference type, but it is not an object.

- Useful for “not available yet” or “no selection” situations.
- Must be handled before invoking a method or reading an instance member.
- A primitive such as int cannot contain null.

Check before use

```
if (currentStudent != null) { ... }
```

NullPointerException: a runtime failure

The source can compile successfully and still fail while executing if you dereference null.

```
Student s1 = null;  
s1.printSummary();
```

runtime output

```
Exception in thread "main"  
java.lang.NullPointerException  
    at Demo.main(Demo.java:4)
```

Compile-time vs. runtime

javac can verify that s1 is a Student reference.

Only at runtime do we discover that its current value is null.

Live demo idea

Run the failing program, read the stack trace, fix with a null check, then run again.

The this reference

Inside an instance method, this refers to the current receiver object.

```
class Student {  
    String name;  
  
    void printName() {  
  
        System.out.println(this.name);  
    }  
}
```

s1.printName()



inside the method

this == receiver s1

When the same method is called on s2, this refers to s2 instead.

Why this matters when names collide

A parameter can have the same name as a field. `this.name` explicitly means the field of the current object.

```
class Student {  
    String name;  
  
    void rename(String name) {  
        this.name = name;  
    }  
}
```

this.name

field belonging to the current object

name

method parameter

```
s1.rename("Ayesha Khan");
```

The method updates the name field of `s1`.

Style point

Using the same parameter name as the field is common in Java. this makes the assignment unambiguous.

Live coding checkpoint 3 — evolve object state through methods

LIVE CODING

Students should see state before and after method calls on two different objects.

Add to Student.java

```
void rename(String name) {  
    this.name = name;  
}  
  
void addCredits(int amount) {  
    this.completedCredits += amount;  
}
```

Demo.java

```
s1.rename("Ayesha Khan");  
s1.addCredits(3);  
s1.printSummary();  
  
s2.printSummary();
```

LIVE CODING

Goal: connect syntax → receiver → current object → state change.

Ask before running

Which object changes? Which object stays exactly as it was?
Explain the answer using the receiver and this.

Second example — CourseSection

A second class confirms that the same ideas generalize beyond Student.

```
class CourseSection {  
    String code;  
    int capacity;  
    int registered;  
  
    boolean hasSeat() {  
        return registered < capacity;  
    }  
  
    void enrollOne() {  
        if (hasSeat()) {  
            registered++;  
        }  
    }  
}
```

State code, capacity, registered

Behavior hasSeat(), enrollOne()

Design conversation

Why is enrollOne() a natural behavior for CourseSection? What data does it need? Notice that the method can directly use the object's own fields.

Preview only

Later lectures will improve this design using private fields, validation, constructors, and stronger invariants.

Objects collaborate by sending method calls

A program is usually a network of objects, not a collection of isolated instances.



We will gradually learn how to assign responsibilities and model these relationships cleanly.

Forward link

Composition, UML relationships, hierarchy design, and interfaces later formalize how objects are connected.

Trace the program — predict before you compile

Tracing references and state is a core skill. Do it on paper before relying on the IDE.

```
Student a = new Student();
a.name = "Ali";

Student b = a;
b.name = "Sara";

Student c = new Student();
c.name = a.name;

System.out.println(a.name);
System.out.println(b.name);
System.out.println(c.name);
```

Your trace table

variable	refers to	name state
a		
b		
c		

Questions

How many Student objects exist? Which variables are aliases? What three lines are printed?

Compile–debug–run challenge

Find the issues before asking the compiler. Then use javac output to verify your diagnosis.

Broken on purpose

```
class Student {  
    int id;  
    String name;  
  
    void printSummary() {  
        System.out.println(id + name);  
    }  
}  
  
public class Demo {  
    public static void main(String[] args) {  
        Student s = new Student()  
        s.id = "101";  
        s.printsummary();  
    }  
}
```

Look for at least three categories

Syntax punctuation / structure

Type value incompatible with declared type

Name wrong identifier / capitalization

Teaching move

Let javac become a second opinion—not the first attempt at thinking.

Common misconceptions to eliminate early

These errors become expensive if students carry them into inheritance and polymorphism.

“The class is the object.”

No. A class defines a type; objects are runtime instances.

“s2 = s1 copies the object.”

No. It copies the reference, creating aliases.

“Same field values means same object.”

No. State can match while identity remains distinct.

“null is an empty Student object.”

No. null means no object reference.

“A compile-successful program cannot fail.”

No. runtime errors such as NullPointerException are still possible.

“this always means the class.”

No. this means the current object receiving an instance-method call.

Lecture recap & exit ticket

Today we moved from the Java execution pipeline to the runtime objects that the program creates and manipulates.

REMEMBER

- Class = type/description; object = runtime instance.
- Fields represent state; methods represent behavior.
- `new` creates an object and returns a reference.
- Reference assignment can create aliases.
- `null` designates no object.
- `this` refers to the current receiver object.

EXIT TICKET

- How many objects are created by two new expressions?
- What happens when `Student b = a;` executes?
- Explain state vs. identity in one sentence.
- What does `this.name` mean inside `rename(String name)`?
- Compile a two-file example without using the IDE Run button.

Next lecture: Data Members & Member Functions — instance variables/methods, overloading, signatures, argument passing, and return types.